

# P3105R0

**constexpr**

**std::uncaught\_exceptions()**

Published Proposal, 2024-01-26

**This version:**

<https://eisenwave.github.io/cpp-proposals/constexpr-uncaught-exceptions.html>

**Author:**

[Jan Schultke](#)

**Audience:**

SG18, LEWG

**Project:**

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

**Source:**

[eisenwave/cpp-proposals](#)

---

## Abstract

This paper marks `std::uncaught_exceptions` and `std::current_exception constexpr`.

## Table of Contents

- 1 Overview
- 2 Motivation
- 3 Possible implementation
  - 3.1 `constexpr uncaught_exceptions`
  - 3.2 `constexpr current_exception`
  - 3.3 `constexpr exception_ptr`

3.4 Non-trivial implementations

3.5 Impact on ABI

## 4 Proposed wording

## 5 Acknowledgements

### References

Normative References

Informative References

## § 1. Overview

I propose to mark the function `std::uncaught_exceptions constexpr`.

Currently, this can be done because there can never be an active exception during constant evaluation.

`std::uncaught_exceptions` would always return zero during constant evaluation. Similarly,

`std::current_exception` would always return a null pointer.

[P2996R1] recommends exceptions as an error handling mechanism for reflections, during constant evaluation. If exceptions were throwable in constant expressions, marking these functions `constexpr` would simply be part of exception support. **However, [P2996R1] is not a prerequisite to this proposal.**

## § 2. Motivation

The motivation is the same as allowing `try-catch` blocks in `constexpr` functions, a feature added to C++20 thanks to [P1002R1]. Allowing the use of `uncaught_exceptions()` in constant expressions makes metaprogramming easier and eliminates special `if constexpr` cases.

A common use case for `std::uncaught_exceptions` is in the implementation of RAII types which execute a function upon destruction, but only if an exception was (not) thrown in the current scope. This is utilized by `std::scope_success` and `std::scope_failure`; see [N4806].

For example, in the implementation of a `constexpr` stack, one may write:

```
constexpr value_type pop() {  
    // Only decrease the size of the stack if no exception was thrown during  
    // copy/move construction of the returned object.
```

```

    // This ensures a strong exception guarantee.
    std::scope_success _{[this] { m_size--; }};
    return std::move(top());
}

```

It is reasonable to mark such code `constexpr`, and ideally `std::uncaught_exceptions` would not be an obstacle to this.

Furthermore, it makes sense to mark `std::uncaught_exceptions`' sister function, `std::current_exception constexpr`. This is done purely for the purpose of consistency. I am not aware of any concrete example of `std::current_exception`'s lack of `constexpr` being an obstacle.

## § 3. Possible implementation

### § 3.1. `constexpr uncaught_exceptions`

```

constexpr int uncaught_exceptions() noexcept {
    if consteval {
        return 0;
    } else {
        return __uncaught_exceptions_impl();
    }
}

```

It is obviously possible for the user to wrap `std::uncaught_exceptions` like this themselves (e.g. [\[ScopeLite\]](#)), but this is an unnecessary burden.

### § 3.2. `constexpr current_exception`

```

constexpr exception_ptr current_exception() noexcept {
    if consteval {
        return exception_ptr(nullptr);
    } else {
        return __current_exception_impl();
    }
}

```

### § 3.3. `constexpr` `exception_ptr`

`std::exception_ptr` would also need to be made a literal type. All major three standard libraries implement `exception_ptr` as a wrapper class for `void*`, which makes this easily possible.

- See [libc++, `exception\_ptr.h`](#).
- See [libstdc++, `exception\_ptr.h`](#).
- See [MSVC STL, `exception`](#).

Simply mark all special member functions `constexpr` and if necessary, guard their implementation with an `if !constexpr` block. It is impossible to create an `exception_ptr` that is not a null pointer during constant evaluations.

### § 3.4. Non-trivial implementations

[P2996R1] suggests allowing `throw` in constant expressions. This would mean that `std::active_exceptions`, `std::current_exception`, and `std::exception_ptr` would no longer have such trivial implementations, and further functions such as `std::make_exception_ptr` may be marked `constexpr`.

The bare minimum compiler support needed for this is:

1. The compiler must track all active exceptions "magically", so that `std::active_exceptions()` returns the correct amount, and `std::current_exception()` returns the current exception. This needs compiler support because such mutable global state normally doesn't exist in constant expressions.
2. `std::exception_ptr` behaves like a type-erased, reference-counted smart pointer. [P2738R1] has been accepted into C++26, adding `constexpr` cast from `void*`. This makes the implementation of such type-erasure in `constexpr std::exception_ptr` feasible.

### § 3.5. Impact on ABI

Multiple functions, including member functions of `std::exception_ptr` would become inline functions if marked `constexpr`. To remain ABI-compatible with existing software, it is necessary to emit these inline function into the runtime library.

libstdc++ already conditionally does this by marking member functions of `std::exception_ptr` `__attribute__((__used__))`. Therefore:

- libstdc++ is not affected.
- MSVC STL already defines all member function of `exception_ptr` as inline functions, and is also not affected.
- libc++ would need to apply similar compatibility measures as libstdc++.

## § 4. Proposed wording

The proposed changes are relative to the working draft of the standard as of [\[N4917\]](#).

Update subclause 17.3.2 [version.syn], paragraph 2 as follows:

```
#define __cpp_lib_constexpr_current_exception 202401L // freestanding,
[...]
#define __cpp_lib_uncaught_exceptions 201411L202401L // freestanding
```

Update subclause 17.9.6 [uncaught.exceptions] as follows:

```
constexpr int uncaught_exceptions() noexcept;
```

Update subclause 17.9.7 [propagation], paragraph 2 as follows:

`exception_ptr` is a literal type ([basic.types.general]) which meets the requirements of *Cpp17NullablePointer* (Table 36). All expressions which must be valid for a *Cpp17NullablePointer* are constant expressions for a null value of type `exception_ptr`.

**NOTE:** This wording is slightly work-in-progress.

Update subclause 17.9.7 [propagation], `current_exception` as follows:

```
constexpr exception_ptr current_exception() noexcept;
```

## § 5. Acknowledgements

The original idea for this paper and a portion of its content have been adopted from a proposal draft by Morwenn.

## § References

### § Normative References

#### [N4917]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 5 September 2022.  
URL: <https://wg21.link/n4917>

### § Informative References

#### [N4806]

Thomas Köppe. *Working Draft, C++ Extensions for Library Fundamentals, Version 3*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4806.html>

#### [P1002R1]

Louis Dionne. *Try-catch blocks in constexpr functions*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1002r1.pdf>

#### [P2738R1]

Corentin Jabot; David Ledger. *constexpr cast from void\*: towards constexpr type-erasure*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2738r1.pdf>

#### [P2996R1]

Wyatt Childers; et al. *Reflection for C++26*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2996r1.html>

#### [ScopeLite]

Martin Moene. *uncaught\_exceptions() wrapper in scope-lite*. URL: <https://github.com/martinmoene/scope-lite/blob/89b274a106363101ea258cb9555a9c6a47ae2928/include/nonstd/scope.hpp#L586-L597>